CSC 2400-002: Design of Algorithms (Spring 2026)

Tennessee Tech University

Drill 14 Solutions

Topic: Decrease and Conquer, Insertion Sort

Question 1. Run the Insertion Sort algorithm on the following array and show the intermediate array after each insertion until the algorithm terminates. I am including the array after the first insertion for you.

Array: [42, 23, 2, 100, 12, 21, 7]

After Insertion 1: [23, 42, 2, 100, 12, 21, 7]

Blank 1 should contain array after Insertion 2, Blank 2 should contain array after Insertion 3, and so on and so forth until Blank 5 contains array after Insertion 6.

Solution: Blank 1 (After Insertion 2): $\boxed{[2, 23, 42, 100, 12, 21, 7]}$

Blank 2 (After Insertion 3): $\boxed{[2, 23, 42, 100, 12, 21, 7]}$

Blank 3 (After Insertion 4): $\boxed{[2, 12, 23, 42, 100, 21, 7]}$

Blank 4 (After Insertion 5): $\boxed{[2, 12, 21, 23, 42, 100, 7]}$

Blank 5 (After Insertion 6): $\boxed{[2, 7, 12, 21, 23, 42, 100]}$

Question 2. What is the best case runtime of Insertion Sort on an n-length array?

Solution: Consider running Insertion Sort on an already-sorted array. Then for each $i = 2, \ldots, n$, the element $A[i]$ is inserted after just one comparison with $A[i-1]$ as $A[i]$ turns out to be bigger. Hence, the total number of comparisons is $n - 1 = \boxed{\Theta(n)}$

Question 3. Write a recursive implementation of Insertion Sort where the first few lines are

```
procedure InsSort(A[1..n])
1: if n = 1 then:
2:     return A
3: InsSort(A[1..(n − 1)])
```

Solution: We only need to add the code for the $i = n$ iteration of the non-recursive implementation.

procedure InsSort($A[1..n]$)
if $n = 1$ **then**:
    return A
InsSort($A[1..(n-1)]$)
$v \leftarrow A[n]$
$j \leftarrow n - 1$
while $j \geq 1$ and $v < A[j]$ **do**:
    $A[j+1] \leftarrow A[j]$
    $j \leftarrow j - 1$
$A[j+1] \leftarrow v$
return A